Java Day 10：面向对象进阶

继承（extends） · 方法重写（Override） · super 关键字

今日目标：理解父类与子类；使用继承复用代码；掌握方法重写与 super；完成性能测试请求结果模型。

一、学习安排

环节	内容	建议时间
1	继承概念与 extends 语法	15 分钟
2	方法重写与 @Override	15 分钟
3	super 与构造方法	15 分钟
4	性能测试场景代码实践	20 分钟
5	独立练习与验收	20 分钟

二、继承（extends）

假设普通 HTTP 请求和登录请求都需要记录响应时间、状态码，并判断是否成功、是否慢请求。若每个类都重复编写这些属性和方法，后续维护会很麻烦。可以把共同部分放到父类中，让子类继承。

父类：保存共性属性和通用方法。子类：继承父类，并增加自己的业务属性或行为。

基本语法：

```
public class RequestResult {
    protected int responseTime;
    protected int statusCode;

    public boolean isSuccess() {
        return statusCode >= 200 && statusCode < 300;
    }
}

public class HttpRequestResult extends RequestResult {
    private String url;
}
```

继承规则

- Java 类只能直接继承一个父类（单继承）。
- 父类 private 成员不能被子类直接访问，但可通过父类提供的 public/protected 方法使用。
- 构造方法不会被子类继承。
- protected 成员可以被子类访问。

三、方法重写（Override）

子类可以重新实现父类的实例方法，以满足自己的业务需求。重写时建议始终添加 @Override，让编译器检查方法签名。

```
public class RequestResult {
    public void showResult() {
        System.out.println("Response Time: " + responseTime);
        System.out.println("Status Code: " + statusCode);
```

```
    }
}

public class LoginRequestResult extends RequestResult {
    private String username;

    @Override
    public void showResult() {
        System.out.println("Username: " + username);
        System.out.println("Response Time: " + responseTime);
        System.out.println("Status Code: " + statusCode);
    }
}
```

重写规则

方法名和参数列表与父类被重写方法一致。

返回类型相同，或符合协变返回类型规则（本阶段先掌握相同类型）。

子类方法访问权限不能比父类更严格。

`private` 方法不能按普通实例方法的方式重写；`static` 方法也不是这里讨论的实例方法重写。

重载与重写的区别

对比	Overload（重载）	Override（重写）
发生位置	通常在同一个类中	父类与子类之间
参数列表	必须不同	必须相同
主要目的	同一功能支持不同参数	子类改变继承来的行为

四、super 关键字

`super` 用于在子类中明确访问父类的构造方法或成员方法。

1）调用父类构造方法

```
public class RequestResult {
    protected int responseTime;
    protected int statusCode;

    public RequestResult(int responseTime, int statusCode) {
        this.responseTime = responseTime;
        this.statusCode = statusCode;
    }
}

public class LoginRequestResult extends RequestResult {
    private String username;

    public LoginRequestResult(int responseTime, int statusCode,
                              String username) {
        super(responseTime, statusCode);
        this.username = username;
    }
}
```

显式调用 `super(...)` 必须是子类构造方法的第一条语句。若未写 `super(...)`，编译器通常尝试插入 `super()`；如果父类没有无参构造方法，子类可能编译失败。

2）调用父类方法

```
@Override
public void showResult() {
    System.out.println("Username: " + username);
    super.showResult();
}
```

这种方式可以保留父类已有逻辑，再追加子类自己的内容，减少重复代码。

五、综合实战：请求结果模型

文件一：RequestResult.java（父类）

```
public class RequestResult {
    private int responseTime;
    private int statusCode;

    public RequestResult(int responseTime, int statusCode) {
        setResponseTime(responseTime);
        setStatusCode(statusCode);
    }

    public int getResponseTime() { return responseTime; }
    public int getStatusCode() { return statusCode; }

    public void setResponseTime(int value) {
        if (value >= 0) responseTime = value;
    }

    public void setStatusCode(int value) {
        if (value >= 100 && value <= 599) statusCode = value;
    }

    public boolean isSuccess() {
        return statusCode >= 200 && statusCode < 300;
    }

    public boolean isSlow() { return responseTime > 500; }

    public void showResult() {
        System.out.println("Response Time: " + responseTime + "ms");
        System.out.println("Status Code: " + statusCode);
        System.out.println("Success: " + isSuccess());
        System.out.println("Slow: " + isSlow());
    }
}
```

文件二：LoginRequestResult.java（子类）

```
public class LoginRequestResult extends RequestResult {
    private String username;

    public LoginRequestResult(int responseTime, int statusCode,
                              String username) {
        super(responseTime, statusCode);
        this.username = username;
    }

    public String getUsername() { return username; }

    @Override
    public void showResult() {
        System.out.println("Username: " + username);
        super.showResult();
    }
}
```

文件三：Main.java（测试类）

```
public class Main {
    public static void main(String[] args) {
        RequestResult normalRequest =
                new RequestResult(200, 200);
        LoginRequestResult loginRequest =
                new LoginRequestResult(850, 500, "Alice");

        System.out.println("=== Normal Request ===");
        normalRequest.showResult();

        System.out.println("=== Login Request ===");
        loginRequest.showResult();
    }
}
```

预期输出

```
=== Normal Request ===
Response Time: 200ms
Status Code: 200
Success: true
Slow: false

=== Login Request ===
Username: Alice
Response Time: 850ms
Status Code: 500
Success: false
Slow: true
```

六、性能测试场景中的应用

性能测试工具或分析平台可能需要表示 HTTP 请求、登录请求、数据库请求、RPC 请求等结果。它们可能共享耗时、状态、成功判断等共性信息，同时又有各自的业务字段。合理抽取父类能集中维护通用逻辑。

设计提醒：不要为了使用继承而继承。两个类只是碰巧有相同字段，并不一定存在合理的父子关系；应先确认它们是否属于同一类概念。

七、今日作业

任务 1：创建父类 ApiResult
包含 private 属性 responseTime、statusCode；构造方法；getter；isSuccess()（200～299）；isSlow()（耗时大于 500ms）；showResult()。

任务 2：创建子类 LoginApiResult
继承 ApiResult；新增 private 属性 username；构造方法中使用 super(...) 初始化父类属性；重写 showResult()，先输出用户名，再调用 super.showResult()。

任务 3：创建 Main 测试类
创建一个普通 ApiResult 对象（例如 180ms、200），再创建一个 LoginApiResult 对象（例如 900ms、500、Alice），分别调用 showResult()，观察输出。

任务 4：思考题
① 为什么子类构造方法要调用 super(...)?
② 如果父类只有带参数构造方法，子类不写 super(...) 会怎样?
③ 重载与重写最关键的区别是什么?
④ 子类重写 showResult() 时，为什么可以使用 super.showResult()?

八、验收标准

- ☐ ApiResult 与 LoginApiResult 能正确编译运行。
- ☐ LoginApiResult 使用 extends 继承父类。
- ☐ 子类构造方法正确调用 super(...)。
- ☐ 重写方法上使用 @Override，签名正确。
- ☐ 子类 showResult() 能输出用户名，并复用父类输出。
- ☐ 能用自己的话解释重载、重写和 super 的区别。

提交方式

完成后上传 ApiResult.java、LoginApiResult.java、Main.java，附上控制台运行结果，并回答 4 道思考题。我会按文件逐项检查；如果条件允许，会实际编译运行后再给出批改意见。

今日完成后，下一课是 Day 11：多态与向上转型，继续把父类引用、子类对象和动态方法调用串起来。